Licenciatura en Sistemas de Información · Arquitectura de Sistemas · Ciclo 2026 · Universidad Champagnat

Trabajo Práctico N.º 3: Despliegue de Arquitectura Cloud-Native

Repositorio Git: https://github.com/constantino280/tp3-globallogistics
Integrantes: Constantino Mateu · Arian Nuñez · Ignacio Azzolina
Fecha de entrega: 02/11/2026 · **Caso:** GlobalLogistics S.A.

1. Resumen de resultados

Desplegamos con `docker compose` un balanceador HAProxy y tres réplicas de un microservicio REST en Node.js. Peticiones consecutivas son atendidas por instancias distintas, y las réplicas que fallan salen de rotación automáticamente.

- **100 usuarios concurrentes:** 3.675 req/s, latencia promedio de 27,3 ms, p95 de 50–52 ms, p99 de 74–81 ms y máximo de 281–365 ms. Cumple el objetivo de tiempo real blando que definimos.
- **2000 usuarios concurrentes:** 4.106 req/s, latencia promedio de 489 ms, p95 de 555–798 ms, p99 de 648–1.292 ms y máximo de 751–1.770 ms. No cumple el objetivo, aunque tampoco hubo errores HTTP.
- **Punto de degradación:** entre 100 y 250 usuarios concurrentes en lazo cerrado, y entre 3000 y 3500 req/s a tasa fija en la mejor medición. El nivel que resultó estable en las dos pasadas de k6 es de 2000 req/s.
- **Tiempo real:** el sistema puede ofrecer tiempo real *blando* por debajo de ese punto, pero no tiempo real *duro*, porque la plataforma no acota la latencia máxima.

2. Arquitectura desplegada

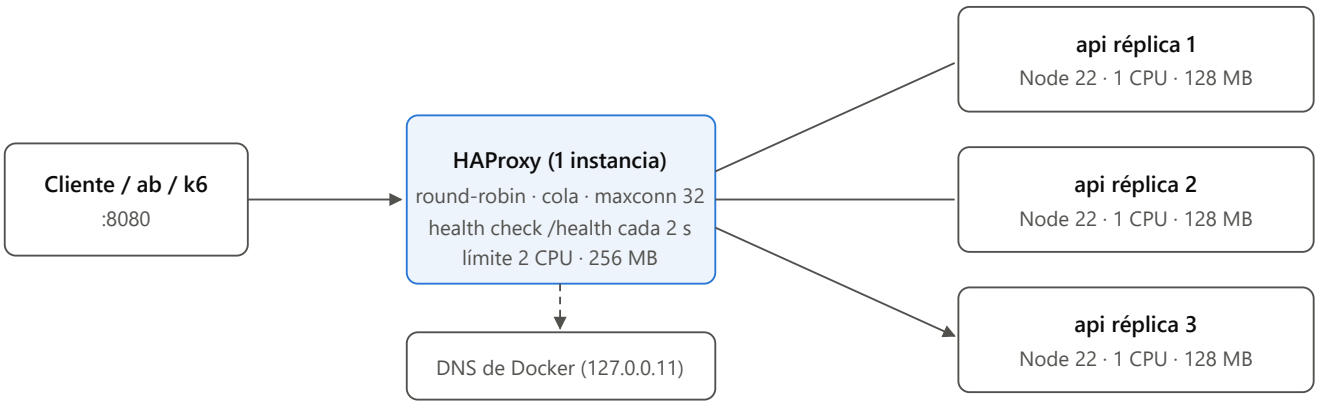


Figura 1. Único punto de entrada publicado: el balanceador. Las réplicas no exponen puertos al host.

Decisión	Justificación
HAProxy en lugar de nginx	El apunte de la Unidad III indica que nginx open source solo tiene health checks pasivos (`max_fails`), que sacan el nodo después de fallar peticiones reales, y que los activos están en NGINX Plus, HAProxy o balanceadores cloud. La consigna pide comprobar el estado de las réplicas, y un check activo lo detecta sin sacrificar tráfico.
Round-robin	Todas las peticiones tienen un costo similar, el supuesto de este algoritmo según el apunte.
Servicio sin estado	Evita sticky sessions: cualquier réplica atiende cualquier petición.
Discovery server-side por DNS de Docker	La infraestructura es efímera: una réplica reiniciada puede cambiar de IP. El código no conoce el registro, la opción más reversible según el apunte.
`maxconn 32` por réplica	Mantiene el exceso en la cola del balanceador. Node atiende de a una petición por vez, así que acumularlas en la réplica también demoraría sus respuestas a `/health`.
Límites de CPU y memoria	Hacen reproducible la saturación e independiente del tamaño del host.

3. Consigna 1: Infraestructura como código

El manifiesto define exactamente un balanceador (`lb`) y tres réplicas (`api`, con `deploy.replicas: 3`). Los generadores de carga usan el perfil `tools`, de modo que `docker compose up -d` no los levanta.

docker-compose.yml

```
name: globallogistics

services:
  # Exactamente un balanceador de carga: único punto de entrada publicado.
  lb:
    image: haproxy:3.0-alpine
    volumes:
      - ./lb/haproxy.cfg:/usr/local/etc/haproxy/haproxy.cfg:ro
    ports:
      - "8080:8080"
      - "8404:8404"
    depends_on:
      api:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "wget", "-q", "-O", "/dev/null", "http://127.0.0.1:8404/stats"]
      interval: 5s
      timeout: 2s
      retries: 3
    deploy:
      resources:
        limits:
          cpus: "2.0"
          memory: 256M
    restart: always
    networks: [backend]

  # Exactamente tres réplicas idénticas. No publican puertos: solo se llega por el balanceador.
  api:
    build: ./api
    image: globallogistics/cotizador:1.0
    # Con init, si el proceso Node muere el contenedor termina y "restart" lo levanta.
    init: true
    environment:
      PORT: "8080"
      WORK_ITERATIONS: "1000"
      CHAOS_ENABLED: "true"
    expose:
      - "8080"
    healthcheck:
      test: ["CMD", "wget", "-q", "-O", "/dev/null", "http://127.0.0.1:8080/health"]
```

```
      interval: 5s
      timeout: 2s
      retries: 3
      start_period: 5s
    deploy:
      replicas: 3
      resources:
        limits:
          cpus: "1.0"
          memory: 128M
    restart: always
    networks: [backend]

  # Generadores de carga: no se levantan con "up", solo con "docker compose run".
  # Corren dentro de la red para no medir el reenvío de puertos de Docker Desktop.
  ab:
    image: httpd:2.4-alpine
    profiles: [tools]
    entrypoint: ["ab"]
    ulimits:
      nofile: 65535
    networks: [backend]

  k6:
    image: grafana/k6:0.54.0
    profiles: [tools]
    volumes:
      - ./scripts:/scripts:ro
      - ./results:/results
    ulimits:
      nofile: 65535
    networks: [backend]

networks:
  backend:
```

Verificación: docker compose ps

```
NAME                    IMAGE                           STATUS                    PORTS
globallogistics-api-1   globallogistics/cotizador:1.0   Up 35 minutes (healthy)   8080/tcp
globallogistics-api-2   globallogistics/cotizador:1.0   Up 8 minutes (healthy)    8080/tcp
globallogistics-api-3   globallogistics/cotizador:1.0   Up 15 minutes (healthy)   8080/tcp
globallogistics-lb-1    haproxy:3.0-alpine              Up 35 minutes (healthy)   0.0.0.0:8080->8080/tcp,
0.0.0.0:8404->8404/tcp
```

4. Consigna 2: Microservicio y balanceo

`GET /` devuelve el hostname del contenedor en el campo `instance`. Cada petición ejecuta 1000 iteraciones de PBKDF2 para simular el costo de CPU de una cotización. Medimos ese cálculo en 0,41–0,62 ms según el momento.

api/server.js

```
'use strict';

// Microservicio de cotización de envíos de GlobalLogistics S.A.
// API REST sin dependencias externas y sin estado: cualquier réplica puede
// atender cualquier petición, requisito para balancear sin afinidad.

const http = require('node:http');
const os = require('node:os');
const crypto = require('node:crypto');

const PORT = Number(process.env.PORT || 8080);
// Iteraciones PBKDF2 que simulan el costo de CPU de calcular una cotización.
const WORK_ITERATIONS = Number(process.env.WORK_ITERATIONS || 1000);
const CHAOS_ENABLED = process.env.CHAOS_ENABLED === 'true';
```

```
const instance = os.hostname();
let forcedUnhealthy = false;

function sendJson(res, status, body) {
  const payload = JSON.stringify(body);
  res.writeHead(status, {
    'Content-Type': 'application/json',
    'Content-Length': Buffer.byteLength(payload),
    'X-Instance-Id': instance,
  });
  res.end(payload);
}

const server = http.createServer((req, res) => {
  const { pathname } = new URL(req.url, 'http://localhost');

  if (req.method === 'GET' && pathname === '/') {
    // Trabajo síncrono a propósito: bloquea el event loop igual que un cálculo
    // intensivo real, de modo que cada réplica sirve de a una petición por vez.
    crypto.pbkdf2Sync('envio', 'globallogistics', WORK_ITERATIONS, 32, 'sha256');
    return sendJson(res, 200, { service: 'cotizador', instance, status: 'ok' });
  }

  if (req.method === 'GET' && pathname === '/health') {
    return forcedUnhealthy
      ? sendJson(res, 503, { instance, status: 'unhealthy' })
      : sendJson(res, 200, { instance, status: 'healthy' });
  }

  // Inyección de fallas para demostrar los health checks. El balanceador
  // bloquea esta ruta: solo se alcanza desde dentro de la red de la réplica.
  if (CHAOS_ENABLED && req.method === 'POST' && pathname === '/chaos/unhealthy') {
    forcedUnhealthy = true;
    return sendJson(res, 200, { instance, forcedUnhealthy });
  }
  if (CHAOS_ENABLED && req.method === 'POST' && pathname === '/chaos/healthy') {
    forcedUnhealthy = false;
    return sendJson(res, 200, { instance, forcedUnhealthy });
  }

  return sendJson(res, 404, { instance, error: 'not found' });
});

// Mayor que el keep-alive del balanceador, para que nunca sea la réplica quien
// cierre una conexión reutilizable mientras el balanceador la está por usar.
server.keepAliveTimeout = 65000;
server.headersTimeout = 66000;

server.listen(PORT, () => {
  console.log(`cotizador ${instance} escuchando en :${PORT} (iteraciones=${WORK_ITERATIONS})`);
});

function shutdown() {
  server.close(() => process.exit(0));
  server.closeIdleConnections();
  setTimeout(() => process.exit(0), 5000).unref();
}
process.on('SIGTERM', shutdown);
process.on('SIGINT', shutdown);
```

lb/haproxy.cfg (balanceador con health checks)

```
global
    maxconn 20000

defaults
    mode http
    option httplog
    option http-keep-alive
    # Si la conexión a una réplica falla, reintenta en otra en lugar de devolver error.
    option redispatch
```

```
    retries 2
    timeout connect 2s
    timeout client  60s
    timeout server  30s
    # Tiempo máximo de espera en la cola del balanceador: pasado este límite responde 503.
    timeout queue   30s
    timeout check   1s

# DNS interno de Docker: resuelve el nombre del servicio "api" a las IP de sus réplicas.
resolvers docker
    nameserver dns1 127.0.0.11:53
    hold valid 2s
    accepted_payload_size 8192

frontend http_in
    bind :8080
    # La inyección de fallas no se expone a los clientes.
    http-request deny deny_status 403 if { path_beg /chaos }
    default_backend cotizador

backend cotizador
    balance roundrobin
    http-reuse safe

    # Health check ACTIVO: sondea /health de cada réplica cada 2 s.
    # 3 fallos seguidos la sacan de rotación; 2 aciertos seguidos la reincorporan.
    option httpchk
    http-check send meth GET uri /health
    http-check expect status 200
    default-server check inter 2s fall 3 rise 2 maxconn 32 resolvers docker init-addr libc,none

    # Descubrimiento server-side: las réplicas se toman del DNS, sin IP fijas.
    # Las 10 ranuras permiten escalar con "--scale api=N" sin tocar esta configuración;
    # con 3 réplicas, las 7 restantes quedan inactivas (MAINT) y no reciben tráfico.
    server-template api- 10 api:8080

listen stats
    bind :8404
    stats enable
    stats uri /stats
    stats refresh 2s
    stats show-legends
```

Evidencia de balanceo

```
$ curl http://localhost:8080/   (x12)
 1  {"service":"cotizador","instance":"c74ccad6f565","status":"ok"}
 2  {"service":"cotizador","instance":"fb342358de8f","status":"ok"}
 3  {"service":"cotizador","instance":"c45a46a7cbfa","status":"ok"}
 4  {"service":"cotizador","instance":"c74ccad6f565","status":"ok"}
 5  {"service":"cotizador","instance":"fb342358de8f","status":"ok"}
 6  {"service":"cotizador","instance":"c45a46a7cbfa","status":"ok"}
 7  {"service":"cotizador","instance":"c74ccad6f565","status":"ok"}
 8  {"service":"cotizador","instance":"fb342358de8f","status":"ok"}
 9  {"service":"cotizador","instance":"c45a46a7cbfa","status":"ok"}
10  {"service":"cotizador","instance":"c74ccad6f565","status":"ok"}
11  {"service":"cotizador","instance":"fb342358de8f","status":"ok"}
12  {"service":"cotizador","instance":"c45a46a7cbfa","status":"ok"}

Reparto de 300 peticiones por instancia:
    100 c45a46a7cbfa
    100 c74ccad6f565
    100 fb342358de8f
```

Figura 2. Peticiones consecutivas al balanceador, atendidas por tres IDs de instancia. De 300 peticiones, cada instancia atendió exactamente 100.

Impresión con sangría

```
{"service":"cotizador","instance":"c74ccad6f565","status":"ok"}
```

Impresión con sangría

```
{"service":"cotizador","instance":"c45a46a7cbfa","status":"ok"}
```

Impresión con sangría

```
{"service":"cotizador","instance":"c74ccad6f565","status":"ok"}
```

Figura 3. Cargas de http://localhost:8080/ en el navegador. No siguen el orden estricto de la Figura 2 porque el navegador también pide /favicon.ico, que consume un turno del round-robin.

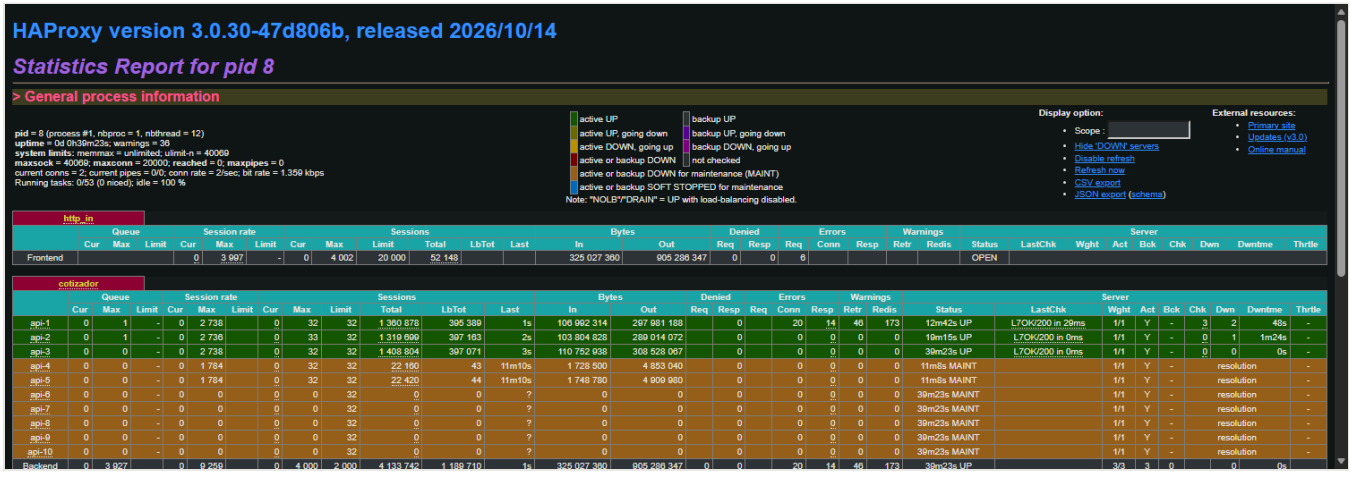


Figura 4. Panel de HAProxy (http://localhost:8404/stats). Las 3 réplicas están activas; las ranuras restantes quedan inactivas hasta que se escale.

Evidencia de health checks y resiliencia

- **Falla de salud:** una réplica que responde 503 en `/health` salió de rotación en 6 s (3 fallos cada 2 s). Mientras estuvo afuera, 30 peticiones se repartieron 15 y 15 entre las otras dos. Al recuperarse, volvió en 4 s.
- **Caída real bajo carga:** matamos el proceso de una réplica con 1.500 req/s en curso. Fallaron 0,022 % de las peticiones (unas 20 de 90.000), el p99 fue de 74,9 ms y el pico de 2.492 ms. Docker reinició el contenedor sin intervención. El reinicio fue tan rápido que la réplica no llegó a figurar como DOWN en el muestreo cada 2 s, así que esos errores probablemente corresponden a peticiones en curso al momento de la caída.
- **Escalado:** con `--scale api=5` el balanceador incorporó las réplicas nuevas sin tocar su configuración, con 10 peticiones por instancia, y al volver a 3 las retiró.

La numeración de las ranuras de HAProxy (`api-1`, `api-2`…) no coincide con la de Compose: en la prueba, la réplica 2 de Compose ocupaba la ranura `api-1`. El registro completo está en `results/resilience.txt`.

5. Consigna 3: Pruebas de estrés y atributos de calidad

5.1 Método

Prueba	Herramienta y parámetros	Qué responde
Escenarios pedidos	`ab -k -c 100` y `-c 2000`, 60 s, 3 repeticiones	Métricas de carga moderada y extrema
Rampa	`ab -k`, de 1 a 4000 usuarios, 20 s por nivel	A qué concurrencia degrada
Tasa fija	k6 `constant-arrival-rate`, 30 s por nivel, 2 pasadas	A qué tráfico real degrada
Escalado	`ab -c 500` con 1, 2, 3 y 5 réplicas	Si agregar réplicas agrega capacidad

Usamos k6 además de ab porque ab trabaja en lazo cerrado: cada usuario espera su respuesta antes de enviar otra petición, sin pausas. Así el generador frena solo cuando el sistema se atrasa y siempre lleva la CPU cerca del 100 %. A tasa fija, en cambio, las llegadas no dependen de la velocidad del sistema. Ambos generadores corren dentro de la red de Docker para no medir el reenvío de puertos de Docker Desktop.

> **Objetivo de tiempo real (supuesto nuestro).** La consigna no fija un valor numérico. Lo definimos para poder evaluar el resultado, tomando como referencia una operación interactiva de logística: **p95 ≤ 100 ms, p99 ≤ 200 ms y máximo ≤ 500 ms**, sin errores. Es un objetivo de tiempo real blando: incumplirlo degrada el servicio, pero no es una falla catastrófica.

5.2 Reporte de métricas

Concurrencia moderada (100 usuarios, 60 s). Tiempos en milisegundos.

Corrida	Peticiones	Req/s	Promedio	p50	p95	p99	Máx.	Errores
1	213.595	3.560	28,1	25	52	81	281	0
2	233.074	3.885	25,7	23	51	81	365	0
3	214.917	3.582	27,9	25	50	74	335	0

Concurrencia extrema (2000 usuarios, 60 s, CPU de réplicas saturada). Tiempos en milisegundos.

Corrida	Peticiones	Req/s	Promedio	p50	p95	p99	Máx.	Errores
1	269.860	4.498	444,7	434	555	648	751	0
2	229.788	3.830	522,2	496	798	1292	1.770	0
3	239.376	3.990	501,3	463	774	977	1.308	0

CPU promedio (`docker stats`, donde 100 % equivale a un núcleo): las réplicas estuvieron entre 92 % y 102 % en ambos escenarios, y el balanceador entre 153 % y 163 % de sus 200 % disponibles. El `ab` usó menos de 40 %, así que no fue el cuello de botella.

5.3 Punto de degradación

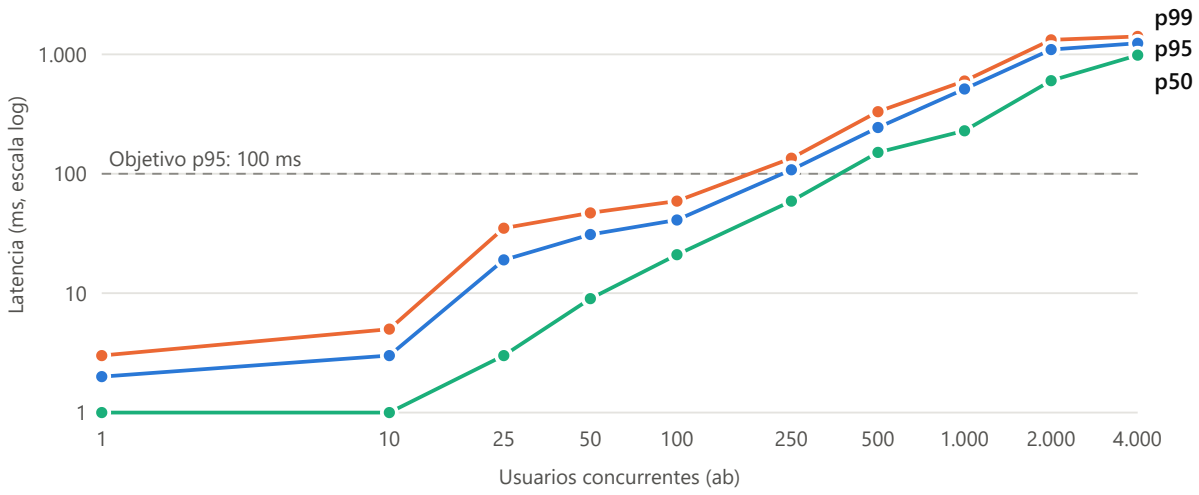


Figura 5. Rampa de concurrencia con ab. Las líneas son percentiles de latencia; la línea punteada es el objetivo de p95.

Usuarios	Req/s	Promedio (ms)	p50	p95	p99	Máx.	Errores
1	861	1,2	1	2	3	79	0
10	6.924	1,4	1	3	5	64	0
25	4.236	5,9	3	19	35	127	0
50	4.234	11,8	9	31	47	173	0
100	4.456	22,4	21	41	59	404	0
250	3.995	62,6	59	108	135	349	0
500	3.120	160,2	151	244	331	653	0
1.000	4.067	245,9	229	513	598	697	0
2.000	2.975	672,2	602	1.097	1.323	1.563	0
4.000	4.002	999,4	985	1.238	1.413	1.971	0

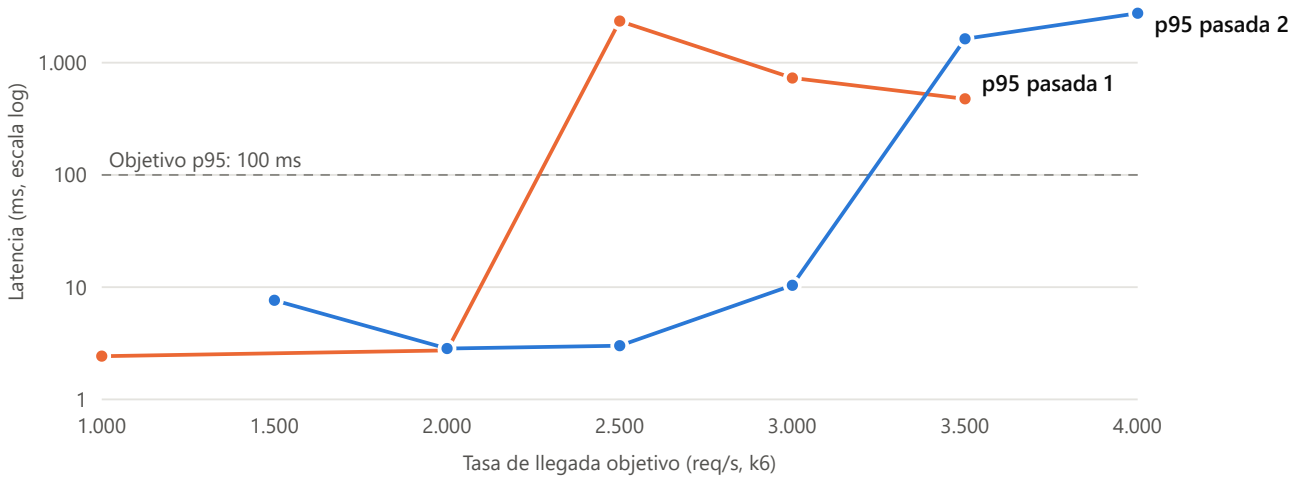


Figura 6. p95 a tasa fija con k6. Las dos pasadas difieren en 2500 y 3000 req/s por la variación del host (ver 5.6).

Objetivo (req/s)	Lograda p.1	p95 p.1	p99 p.1	Máx. p.1	Lograda p.2	p95 p.2	p99 p.2	Máx. p.2
1.000	1.000	2,4	4,2	59	–	–	–	–
1.500	–	–	–	–	1.500	7,6	29,6	339
2.000	2.000	2,7	8,0	70	1.998	2,8	9,4	147
2.500	1.432	2.349,5	2.780,9	3.289	2.491	3,0	55,6	271
3.000	2.642	730,2	911,1	1.065	2.991	10,4	65,0	433
3.500	3.222	476,6	614,4	866	2.469	1.632,2	1.845,8	2.369
4.000	–	–	–	–	2.198	2.758,0	2.834,3	3.030
5.000	3.744	1.639,5	1.692,6	2.300	–	–	–	–
6.000	3.152	1.734,0	1.824,6	1.946	–	–	–	–
6.500	3.012	1.780,4	1.935,6	2.215	–	–	–	–
7.000	2.755	2.097,9	2.202,4	2.336	–	–	–	–
8.000	2.327	2.327,6	2.369,8	2.522	–	–	–	–

Tiempos en milisegundos. No hubo peticiones fallidas en ningún nivel, pero sobre la capacidad k6 descartó llegadas que no pudo iniciar, por lo que esas latencias subestiman la espera real.

5.4 Análisis crítico

¿En qué punto degrada? Por concurrencia, el p95 pasa de 41 ms con 100 usuarios a 108 ms con 250, donde supera el objetivo, y a 244 ms con 500. Por tráfico, la mejor pasada se mantiene con p95 de 10,4 ms a 3000 req/s y salta a 1.632 ms a 3500 req/s. El quiebre es abrupto, como predice la teoría de colas cuando la utilización se acerca a 1. La primera pasada ya había fallado el objetivo a 2500 req/s, mientras que 2000 req/s fue estable en las dos, así que ese es el nivel que recomendamos como límite operativo.

¿Cumple las restricciones de tiempo real?

Escenario	p95 ≤ 100 ms	p99 ≤ 200 ms	Máx. ≤ 500 ms	Errores
100 usuarios (peor de 3 corridas)	52 ms ✓ Cumple	81 ms ✓ Cumple	365 ms ✓ Cumple	0
2000 usuarios (peor de 3 corridas)	798 ms ✗ No cumple	1.292 ms ✗ No cumple	1.770 ms ✗ No cumple	0

Con 100 usuarios el máximo cumple, pero con poco margen: en corridas de control de 10 s llegó a 508 ms (`results/control_runs.txt`), apenas por encima del objetivo. Además, el máximo fue de 3,5 a 4,5 veces el p99, de modo que la cola no está acotada. Este despliegue puede sostener tiempo real blando con carga

moderada, pero no tiempo real duro: Linux, el planificador de CPU de los contenedores, el recolector de basura de Node y la red virtual no ofrecen ninguna garantía de peor caso.

¿Cómo se justifica por la arquitectura?

- **El cuello de botella son las réplicas.** Node ejecuta el cálculo en su único hilo principal y cada réplica tiene 1 CPU. En ambos escenarios las réplicas operan cerca del 100 %, por lo que la capacidad queda fija en unas 3000–4500 req/s en estas corridas y lo único que crece con los usuarios es la espera. Con 4.106 req/s y 2000 usuarios, una espera promedio de unos 489 ms es lo que anticipa la ley de Little.
- **La cola está en el balanceador.** Con `maxconn 32`, cada réplica recibe como máximo 32 peticiones y el resto espera en HAProxy, donde observamos hasta 3927 en cola. Por eso la saturación se manifiesta como latencia y no como errores: 0 errores en 739.024 peticiones del escenario extremo.
- **Los health checks resistieron la saturación.** Al terminar las pruebas de carga, ninguna réplica registraba checks fallidos (`chkfail=0`). Las réplicas deben responder `/health` en menos de 1 s, y lo consiguen porque esperan como mucho unas 32 peticiones. Sin ese límite, la saturación podría hacer que el balanceador las diera de baja y empeorar la situación en cascada.
- **La escala horizontal ayuda hasta cierto punto.** Como muestra la Figura 7, pasar de 1 a 2 réplicas duplicó el rendimiento (×2,06), pero de 3 a 5 solo sumó ×1,25. Con 5 réplicas el balanceador ya usa 172 % de sus 200 %, y todo compite por el mismo host: el cuello empieza a moverse fuera de las réplicas, el límite que anticipa el apunte. El rendimiento con 2 y 3 réplicas (×1,11) también puede estar afectado por la variación del host.
- **Por encima de la capacidad, el rendimiento cae.** A tasa fija pasa de 3.744 req/s con 5000 de objetivo a 2.327 con 8000. Nuestra hipótesis, que no verificamos, es que k6 y HAProxy le quitan CPU a las réplicas en el mismo host, ya que k6 llegó a usar más de 4 núcleos.

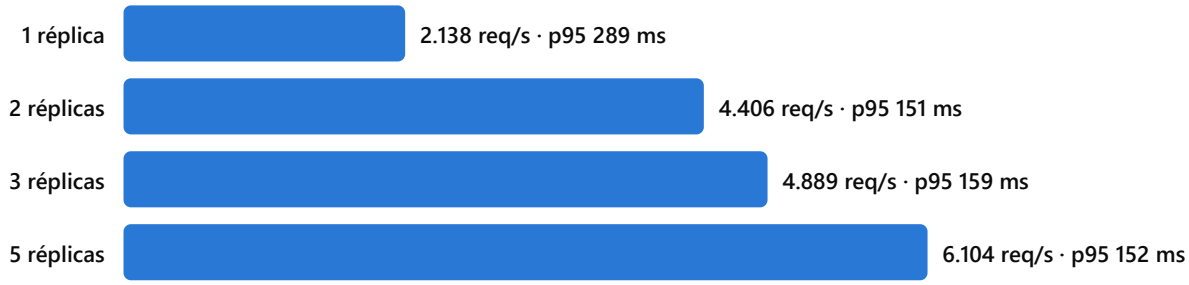


Figura 7. Rendimiento con 500 usuarios según la cantidad de réplicas (una corrida de 20 s por nivel).

5.5 Recomendaciones

- **Descartar antes de esperar:** bajar `timeout queue` de 30 s a un valor cercano al objetivo (por ejemplo, 500 ms) para responder 503 rápido en lugar de hacer esperar segundos. No lo probamos.
- **Autoescalado real:** Compose no escala solo. Con estas pruebas demostramos que el balanceador incorpora réplicas sin reconfigurarse, pero la decisión sigue siendo un comando. En producción la tomaría Kubernetes HPA o ECS, disparando antes del quiebre: cerca de 670 req/s por réplica (2000 req/s entre 3).
- **Eliminar el punto único de fallo:** el balanceador único pedido por la consigna es un SPOF. Se mitiga con un par activo/pasivo y VRRP, o con un balanceador gestionado.
- **No bloquear el event loop:** pasar el cálculo a `worker_threads` aprovecharía réplicas con más de una CPU.

5.6 Limitaciones de la medición

- **Host compartido y a batería:** todo corrió en una notebook con Intel i5-12450H (8 núcleos, 12 hilos) y 8 GB para Docker sobre WSL2, a batería, con carga entre 29 % y 21 % en la parte final, y con otras aplicaciones abiertas. La capacidad varió mucho: el costo de CPU por petición fue de 0,41 a 0,62 ms en

una misma réplica, y hubo picos de 6924 req/s con 10 usuarios y otras corridas de 2500 req/s con la misma concurrencia. **Por eso los números deben leerse como rangos y no como capacidad absoluta.** La anomalía de la primera pasada de k6 a 2500 req/s es atribuible a este ruido, aunque no pudimos aislar la causa.

- **Generador en el mismo host:** ab y k6 compiten por CPU con el sistema medido.
- **Resolución de ab:** reporta percentiles en milisegundos enteros, poco preciso con cargas bajas.
- **Lazo cerrado:** ab no refleja las peticiones que un cliente real habría enviado durante una demora (*coordinated omission*), por eso sumamos k6.
- **Muestras de una corrida:** la rampa y el escalado tienen una sola corrida por nivel.
- **Carga sintética:** el servicio no usa base de datos ni red externa. Con esas dependencias, el cuello podría estar en otro lugar y las réplicas adicionales podrían no ayudar.

6. Conclusión

La arquitectura cumple lo pedido: un manifiesto levanta 1 balanceador y 3 réplicas, el tráfico se distribuye de forma uniforme y una réplica que falla sale de rotación y se recupera sin intervención. Con 100 usuarios concurrentes los tiempos de respuesta se mantienen dentro del objetivo de tiempo real blando, aunque con poco margen en el máximo. A partir de unos 250 usuarios, o de 3000–3500 req/s, la latencia se dispara porque la CPU de las réplicas se agota y las peticiones esperan en el balanceador. El sistema no pierde peticiones, pero deja de responder a tiempo: gana disponibilidad a costa de latencia, por lo que necesita un límite de cola y autoescalado para sostener el objetivo bajo estrés comercial.

Anexo: reproducción

El detalle está en el README del repositorio. Los resultados crudos de cada corrida están en `results/raw/`.

```
docker compose up -d --build --wait
./scripts/test-balancing.sh
./scripts/test-resilience.sh
./scripts/run-load-tests.sh
```